Adapting AlphaEvolve to Optimize Fully Homomorphic Encryption on TPUs

Shruthi Gorantala*, Jianming Tong‡*, Baiyu Li*,
Asra Ali*, Jeremy Kun*, Thomas Steinke†, Abhradeep Thakurta†, Julian Walker†, Amir Yazdanbakhsh†,
Jonathan Katz*
*Google †Google DeepMind ‡Georgia Institute of Technology

***Abstract*—The deployment of Fully Homomorphic Encryption (FHE) at scale is hindered due to its heavy computational overhead. While specialized hardware accelerators like Google Tensor Processing Units (TPUs) can help, mapping complex cryptographic kernels onto such architectures remains a challenge. Efficient execution requires co-optimization between the systolic array-based Matrix Multiplication Unit (MXU) and Vector Processing Units (VPUs), as well as the orchestration of data movement across the vector register files. Existing compiler stacks often abstract low-level hardware utilization, requiring developers to adopt a manual trial-and-error process that often results in fragmented execution and underutilized resources.**

To accelerate this development process, we use AlphaEvolve to automate the exploration of hardware-aware cryptographic-kernel optimizations. We frame optimization as an evolutionary search problem, utilizing the closed-loop system provided by AlphaEvolve, that leverages LLM-driven code generation. We use real-world feedback from hardware execution and rigorous correctness testing to guide the evolution process. We evaluate AlphaEvolve optimization on primitives for both the TFHE (Jaxite) and CKKS (CROSS) FHE schemes on Google Cloud TPUv5e, a contemporary TPU architecture. Within 24 hours of automated exploration, AlphaEvolve discovered implementation-level optimizations that improve TFHE bootstrapping latency by 2.5× and CKKs rotation and multiplication latency by 1.30× and 1.18×, respectively, relative to human-engineered state of the art. These results demonstrate that AlphaEvolve can be used as a productivity-boosting tool, enabling researchers to navigate the trade-offs between cryptography, compilers, and hardware accelerators.

I. Introduction

The rapid proliferation of personal data in AI ecosystems has created a demand for privacy-enhancing technologies (PETs). Fully Homomorphic Encryption (FHE) [7] stands as a foundational Privacy Enhancing Technology (PET) by enabling arbitrary computations on encrypted data without requiring decryption. Despite its privacy guarantees, Fully Homomorphic Encryption (FHE) incurs orders-of-magnitude compute and memory overheads, severely hindering its scalable deployment [8]. To mitigate these bottlenecks, the community has increasingly pivoted toward specialized accelerators like Tensor Processing Units (TPUs).

Mapping FHE primitives to accelerators requires engineering optimizations that spans across tiling workloads to strictly align with the underlying vector register granularity, navigating frontend abstractions (e.g., JAX and Pallas APIs for TPUs) to bypass costly, implicit layout transformations and sizing tensor granularities to effectively reduce off-chip memory latency. Modern compiler stacks like XLA [17] abstract these microarchitectural details. However performance tuning then becomes a slow trial-and-error tuning loop where high-level code modification must traverse the compilation pipeline to observe its hardware impact. A critical knowledge gap persists in how to systematically map transformed cryptographic primitives onto commodity hardware designed for low-precision, GEMM-heavy workloads.

AI coding agents such as AlphaEvolve [15] have demonstrated significant advances in multiple fields including algorithm design, improving code efficiency, architecture discovery, and enabling discovery across scientific domains. Further, the use of AI to improve cryptographic implementation remains largely unexplored. This is primarily because Large Language Model (LLM) hallucinations might introduce security vulnerabilities which are unacceptable in cryptographic implementations.

In this paper, we use AlphaEvolve, an agentic framework, to optimize FHE kernels on hardware accelerators, such as Google's TPU, with rigorous correctness checking. We formulate kernel optimization as an evolutionary search problem using AlphaEvolve, with real-world hardware feedback. By automating the discovery of micro-architectural improvements, AlphaEvolve enables cryptographic developers to explore architectural ideas and co-optimize cryptographic primitives on specialized hardware.

The main contributions of our work are:

- We utilize wall-clock latency from real hardware execution traces (XProf) as a primary reward signal to AlphaEvolve. This enables the coding agent to algorithmically identify microarchitectural bottlenecks, such as implicit data transformation and explicit off-chip memory loading overhead, and perform iterative performance-improving edits.
- We implement robust evaluation harnesses that enforce strict multi-tiered checks. These include randomized input vector correctness tests across computational sub-layers and 128-bit security checks using industry-standard tools to ensure that performance gains are achieved while maintaining security guarantees.
- We demonstrate that AlphaEvolve independently discovers implementation-level non-trivial edits, such as mixed-precision safe downcasting to bfloat16 and targeted loop unrolling, that outperform domain-expert baselines. Within 24 hours of exploration, AlphaEvolve achieved a 2.5×

reduction in TFHE bootstrapping latency (from 10ms to 4ms) and speedups of 1.30× and 1.18× for CKKS rotation and multiplication, respectively.

II. Background

A. AlphaEvolve

AlphaEvolve [15] is a coding agent designed for algorithmic and scientific discovery by combining LLMs with evolutionary search techniques. It enables the automated evolution of code files in any programming language through iterative code generation and code evaluation.

The workflow begins with a human programmer configuring the system with instructions (background knowledge specific to the task), an initial code to seed the evolutionary database and an evaluator that scores the generated code.

Next, the system samples existing code blocks and instructions to construct specialized prompts. These prompts are fed into LLMs to generate new code blocks. The generated code blocks are then compiled and fed into the evaluators, where they are scored, ranked, and best programs are added back to the evolutionary database. The ranking and evolution employs structured evolutionary techniques, such as island-based models or MAP-Elites algorithms [15]. These techniques balance genetic diversity based on inherent metrics (e.g., program length, code complexity) and performance on the evaluator metrics (fitness score). This optimization loop automatically improves the code to find the best program based on the fitness score.

B. Fully Homomorphic Encryption

Fully homomorphic encryption schemes [7] enable computation on encrypted data. A homomorphic operation modifies the encrypted data (i.e., ciphertext) so that the decrypted result is the same as it would have been if the operation were performed on the unencrypted data. As all computations are performed on encrypted data, FHE provides holistic security guarantees against side-channel attacks and hence is chosen as the first case study of applying AlphaEvolve into cryptography.

FHE encryption schemes can be broadly categorized into scalar schemes and vector schemes based on the type of data they encrypt. Scalar schemes encrypt bits or short integers, such as the TFHE scheme [6], and supports general-purpose computation on encrypted data, while the CKKS scheme [5] operates on vectors of integers or floats and offers higher performance for machine learning and numerical computations on encrypted data as vectors of data are processed in parallel.

In this paper, AlphaEvolve is used to optimize primitives from both FHE schemes on TPUs.

1) `TFHE-bootstrap`: the key primitive in TFHE scheme, implemented in jaxite library [2]. It's the core primitive used in Google transpiler [9] and HEIR compiler [3]. We give further details in §IV-A.

2) `CKKS-Rot` and `CKKS-Mult`: kernels in TPU-based CKKS library, CROSS [19], used for private ML inference.

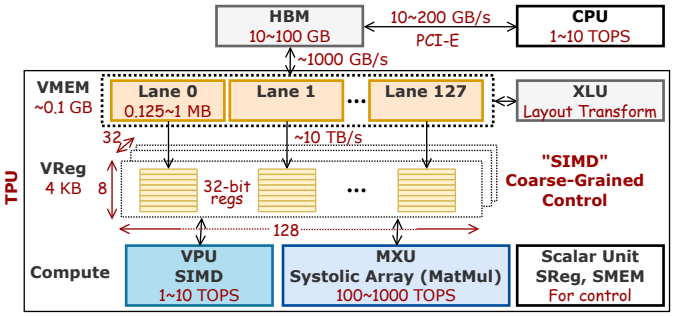

Fig. 1: TPU Micro-architecture.

C. Google's TPU

AI ASICs, such as Google's TPUs [13], deliver exceptional throughput and energy efficiency [18], [19]. However, achieving peak performance requires strictly aligning algorithms with the TPU's highly rigid hardware architecture.

The TPU's compute and memory hierarchy is heavily optimized for vectorized processing. On-chip memory is organized into Vector Memory (VMEM) divided across 128 lanes—each containing 8 sublanes—feeding thousands of local ALUs in the Vector Processing Unit (VPU). Crucially, these ALUs operate in strict lock-step using Single Instruction, Multiple Data (SIMD) instructions to process data across vectorized registers (VRegs) that are strictly shaped as an (8, 128) matrix.

Because computation and data routing are structurally bound to this massive (8, 128) VReg granularity, the TPU architecture heavily punishes fine-grained, slot-wise data reordering and scalar rotations. Even dedicated hardware for on-chip data reorganization, such as the Cross Lane Unit (XLU), operates optimally at this full VReg level. Therefore, the critical **takeaway** for TPU optimization is that **programs must be heavily reformed away from slot-wise manipulations and restructured entirely into coarse-grained, block-level operations that natively fit the (8, 128) VReg shape.**

D. The TPU Compiler Stack

We inherit the multi-layer TPU compilation stacks and profiling system to support FHE computations:

1) JAX: acts as the high-level frontend framework for numerical computing. FHE libraries such as Jaxite [2] and CROSS [19] use JAX programming language to express cryptographic logic, enabling efficient integration with downstream compilers [4].

2) XLA: is the domain-specific compiler backend utilized by JAX. It takes the computational graphs generated by JAX and compiles them into highly optimized machine code specifically targeted for accelerators like TPUs and GPUs. XLA automatically handles general performance optimizations such as operator fusion [10].

3) Pallas: serves as a kernel language that allows for the creation of custom, low-level hardware kernels within JAX. Our system relies on Pallas kernels to establish fine-grained control over data movement and memory staging on the TPU [12].

4) Xprof: is an integrated profiling tool within the XLA ecosystem that captures high-fidelity hardware execution traces. It extracts metrics such as compute and memory bandwidth utilization as well as on-chip and off-chip storage occupancy. We feed Xprof-based latency as critical diagnostic signals back into the evolutionary search loop, enabling performance-oriented kernel refinements [16].

III. AlphaEvolve - Adapting AlphaEvolve for Cryptography

Configuring AlphaEvolve requires users to specify initial prompts (which establish a persona, algorithm and hardware platform), initial programs (which will be optimized by AlphaEvolve), the AlphaEvolve framework itself and an evaluation strategy (which validates the correctness, and then feeds reward as feedback back to the AlphaEvolve).

Fig. 2 provides an overview of various parts of the system with details described below:

A. Initial Prompts

The initial prompts describe the task and role of AlphaEvolve as an expert in hardware-aware kernels engineering. The prompts also include a functional description of the cryptographic algorithms being optimized, architecture of the hardware accelerator, optimization strategies and a pool of potential optimization strategies. The AlphaEvolve framework is configured to update these prompts from metadata from previous runs, to guide the search for performant kernels gathering feedback from the evaluators described below.

B. Initial Program

We feed well-optimized reference codes into the evolutionary search framework as the initial programs. AlphaEvolve, then invokes the LLM code generator to rewrite this code with potential optimizations. The choice of initial program needs a balance between providing enough code for AlphaEvolve to optimize while not allowing it to change the security guarantees provided by the algorithm. We use co-evolution as a process to evolve multiple kernels in tandem to find kernel improvements within various aspects of the cryptosystem. In addition to co-evolution, co-optimization can be used to evolve security parameters and kernels, which allows for tuning security parameters for accelerator-specific kernel super-optimization.

C. Evaluation Engine

Central to the our design is a robust evaluation engine that steers the evolutionary search towards valid programs. We employ both correctness checks and security checks to fail invalid and insecure code.

1) Correctness Checks: To mitigate the risks of reward hacking and code hallucination inherent in LLM-generated kernels, we enforces strict correctness tests using randomized input vectors across all computational sub-layers. The correctness checking requires correct compilation to mitigate the risks of unintended bugs and erroneous code. This is done by testing subset of sub-modules as well as incorporating end-to-end tests which initializes the cryptosystem with valid security parameters, generates randomized inputs, encrypts the inputs, optionally runs cryptographic computations on the ciphertexts such as homomorphic computations for FHE cryptosystem, decrypts the resultant ciphertexts and checks against the expected plaintext outputs.

2) Security Checks: Our system validates any modifications to the parameters of the cryptosystem maintain standardized security guarantees (example: at least 128-bit security for Fully Homomorphic Encryption utilizing acceptable parameters generated from industry standard lattice estimators [1]). Furthermore in case of fully homomorphic encryption, we implement safeguards to prevent the selection of scheme parameters that could lead to decryption failures by adding additional end-to-end test with high depth computation. The evaluation engine only rewards candidate kernels that satisfy these multi-tiered correctness and security checks, thereby pruning invalid or insecure code early in the evolutionary search process.

3) Target Hardware Latency as Reward: By deploying directly on the target platform, we use real-world latency as the primary reward score. In this paper, candidate kernels in jaxite are lowered through the XLA compiler and executed on physical TPU hardware. Our reward score uses microsecond-level execution latency of the resulting cryptographic primitive, excluding XLA compilation overhead to isolate runtime performance.

A persistent bottleneck in FHE acceleration is the lack of parity between isolated primitive speedups and actual end-to-end performance due to the bottlenecks introduced by data movement between host and device and also between devices. To ensure primitive kernel speedup translates to end-to-end speedup, AlphaEvolve evolves the one or more primitive kernels simultaneously, while using end-to-end latency across the entire functional module as the score. Measuring end-to-end latency proves to be a useful reward metric for co-evolution and co-optimization techniques.

4) Feedback: To facilitate the autonomous discovery of hardware-optimal execution patterns, our system captures high-fidelity execution traces during the evaluation cycle using xprof integrated with XLA. These metrics from profiling including compute and memory bandwidth utilization as well as on-chip and off-chip storage occupancy are fed back to AlphaEvolve as diagnostic feedback. By analyzing these traces, the agent can algorithmically identify latency bottlenecks and instances where the XLA compiler needs additional signal through JAX (or Pallas), enabling targeted iterative refinements to the cryptographic kernels.

D. Human reviews / Functional equivalence

The best program generated by the evolutionary loop can be integrated into the original library after human review. Currently, a human programmer reviews the best program and merges the improved code into the original library. To reduce human-in-the-loop reviews, a module to check functional

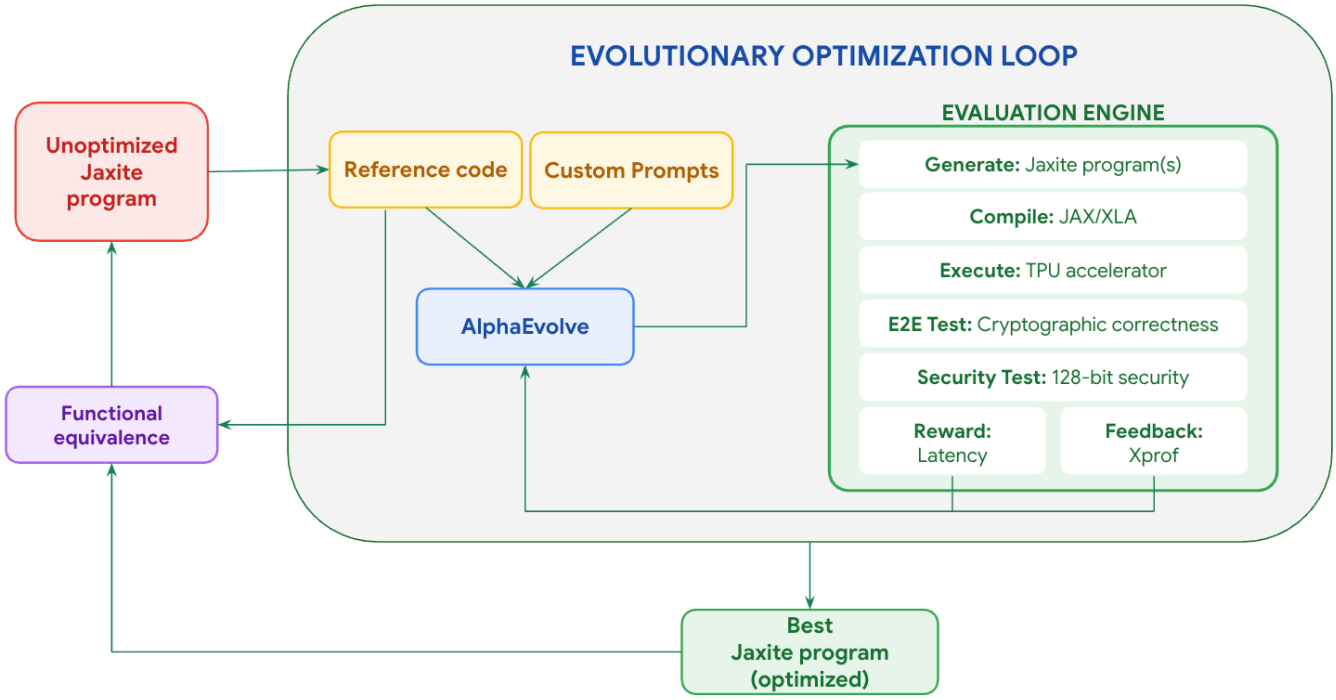

Fig. 2: Adapting AlphaEvolve for optimizing cryptographic primitives. The evolutionary optimization loop is driven by AlphaEvolve using reference code, custom prompts and evaluation engine. The best generated program is then integrated into the original jaxite program after a functional equivalence check and human reviews.

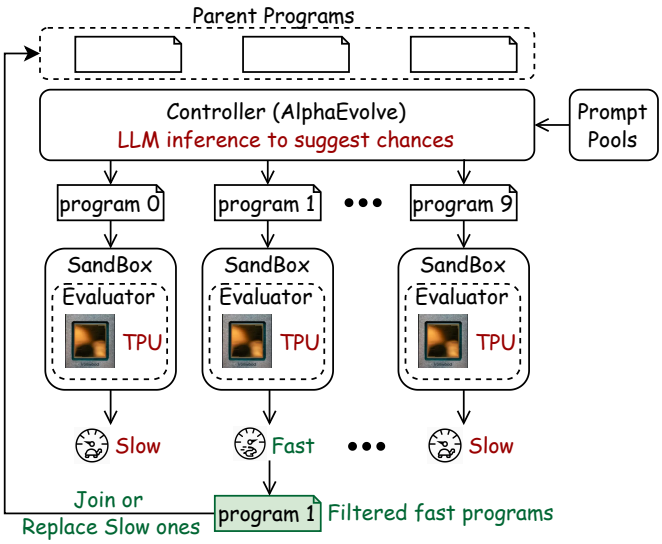

Fig. 3: AlphaEvolve Evaluation Engine

equivalence of the reference and AlphaEvolve generated code can be used.

E. Putting it all together

After configuring custom prompts, reference code, evaluation engine, and setting up appropriate hardware environment for the evaluators, AlphaEvolve begins the discovery process of automatically optimizing the cryptographic kernels. After reaching convergence based on the fitness function, the best generated program can be further tested for correctness through human reviews. The final code can then be merged into the original library and deployed for use.

IV. AlphaEvolve for optimizing TFHE bootstrap

TFHE [6] is a scalar FHE scheme supporting arbitrary homomorphic operations on encryptions of boolean and small integer values. **Bootstrap** in TFHE is not only the process to refresh the ciphertext noises but it is essentially the core component to support most nontrivial homomorphic gate operations.

A. Overview of TFHE Bootstrapping

In TFHE, a ciphertext encrypting a scalar m is essentially of the form $(\mathbf{a}, b) \in \mathbb{Z}_q^{n+1}$; to decrypt it, the secret key holder computes $\mathsf{round}(b - \langle \mathbf{a}, \mathbf{s} \rangle \pmod q) = m$, where q is a power of 2, $\mathbf{s} \in \{0,1\}^n$ is the underlying secret vector, and round is a rounding function that is capable of removing the ciphertext noise. The bootstrap algorithm takes as input the ciphertext $C = (\mathbf{a}, b)$ and an encryption $E(\mathbf{s})$ of $\mathbf{s}$ (under possibly another homomorphic encryption scheme), and homomorphically computes this decryption operation to obtain a new ciphertext C' encrypting the same message m but with less noise. The second encryption scheme E is instantiated using the so-called RingGSW encryption defined

over a polynomial residual ring $R_Q = \mathbb{Z}_Q[X]/(X^{q/2}+1)$, and the bootstrapping key $E(\mathbf{s})$ consists of n ciphertexts $E(s_1), \ldots, E(s_n)$ encrypting individual components of $\mathbf{s}$. TFHE employs a special RingGSW ciphertext ACC to homomorphically perform such decryption computation, and in particular it enables more general table lookup $f : \mathbb{Z}_q \to \mathbb{Z}_t$ than a simple reduction modulo q of its underlying value. The special structure of R_Q allows us to represent $v \in \mathbb{Z}_q$ using a monomial $X^v \in R_Q$, and hence the decryption computation $\langle \cdot, \mathbf{a} \rangle \bmod q$ can be carried out on the exponent using only multiplication and addition. Since multiplying a polynomial $g(X)$ with a monomial X^{-a} in R_Q effectively rotates g's coefficients by $a \pmod q$, the constant term of ACC at the end of bootstrapping encodes the refreshed LWE ciphertext. The resulting bootstrap process can be described as an algorithm in Figure 4.

Algorithm: $\mathsf{Bootstrap}(C, \mathsf{bsk})$
Input: $\mathsf{bsk} = (E(s_i))_{i \in [n]}$, $C = (\mathbf{a}, b)$

// Init ACC *with table entries* $f(b-i)$
$\mathsf{ACC} = \sum_i f(b-i) X^i$
// Blind rotation: homomorphically compute $f(\sum -a_i s_i)$
for $i = 1$ to n:
  $\mathsf{ACC} = \mathsf{ACC} + (X^{-a_i} - 1) \cdot \mathsf{ACC} \cdot E(s_i)$
// Extract LWE encryption under secret z
$C'' = \mathsf{Extract}(\mathsf{ACC})$
return $\mathsf{KeySwitch}_{z \to \mathbf{s}}(C'')$

Fig. 4: Outline of TFHE bootstrap algorithm. The lookup table f must satisfy $f(v + q/2) = -f(v)$ for all $v \in \mathbb{Z}_q$.

The primary bottleneck in the bootstrap operation is `blind_rotate`, which consists of a sequential multiplication-accumulation of a matrix-vector product of polynomials. Here polynomial multiplication is reformulated as toeplitz matrix multiplication to take advantage of MXUs in TPUs.

Algorithm: $\mathsf{ToeplitzMul}(\mathbf{a}, \mathbf{b})$
Input: $\mathbf{a} = (a_0, \ldots, a_{n-1})$, $\mathbf{b} = (b_0, \ldots, b_{n-1})$

Construct the Toeplitz matrix $T(\mathbf{a}) \in \mathbb{Z}^{n \times n}$:

$$T(\mathbf{a}) = \begin{bmatrix} a_0 & a_1 & \cdots & a_{n-1} \\ -a_{n-1} & a_0 & \cdots & a_{n-2} \\ \vdots & \vdots & \ddots & \vdots \\ -a_1 & -a_2 & \cdots & a_0 \end{bmatrix}$$

// Note: Negacyclic entries account for reduction modulo $X^n + 1$
Initialize result vector $\mathbf{c} = (0, \ldots, 0)$ of length n
for $i = 0$ to $n-1$:
  $c_i = \sum_{j=0}^{n-1} T(\mathbf{a})_{i,j} \cdot b_j$
  // Compute the i*-th coefficient of the product polynomial*
return $\mathbf{c} = \sum_{i=0}^{n-1} c_i X^i$

Fig. 5: Polynomial multiplication via Toeplitz matrix-vector product. This specific form represents multiplication in the cyclotomic ring $R_q = \mathbb{Z}_q[X]/(X^n+1)$.

B. Setup

AlphaEvolve pipeline is configured as follows:

1) Prompt: AlphaEvolve is set up with the prompts of role of computer architect and TPU architecture. The comprehensive specification is provided in the Appendix.

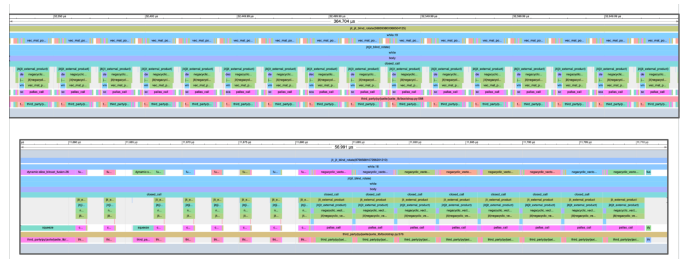

Fig. 6: AlphaEvolve improvement. Bootstrap trace before and after evolution (top: 10 ms, bottom: 3 ms)

2) Reference code: we use the hand optimized implementation of bootstrap in **Jaxite** [2]. For this case study, we restrict AlphaEvolve to only modify the `blind_rotate` and `external_product` modules .

3) Evaluator Score: we use the latency of bootstrap on fixed 128-bit security parameters to ensure co-evolution of sub-modules results in end-to-end speedup.

4) Correctness Checks: we check the exhaustive space of 3-bit lookup tables for correctness on randomly generated 128-bit security parameters. Each bootstrap operator takes about 10 ms to execute and there are 256 values to be tested.

C. Results

1) Microbenchmark – CGGI Bootstrapping on TPU: On a physical TPU v5e (1×1 topology), the kernel generated by AlphaEvolve executes 4-bit CGGI bootstrapping in 4 ms. This achieves a 2.5× speedup over the 10 ms baseline of state-of-the-art, human-optimized TPU implementations. The 4 ms result demonstrates that AlphaEvolve effectively makes the TPU faster than traditional CPU execution for FHE workloads

2) End-to-End Speedup – Encrypted Regex Redaction: To conclusively prove end-to-end viability, the AlphaEvolve-generated primitive was integrated into a encrypted string redaction program and the primitive speedup translates into end-to-end speedup.

D. Analysis of AlphaEvolve improvements

To understand the mechanisms behind the performance gains achieved by AlphaEvolve, we analyzed the specific code transformations generated by the underlying AlphaEvolve framework during the optimization of the FHE bootstrapping kernel. The system explored several distinct classes of optimizations, ranging from standard loop transformations to complex kernel scheduling and data type optimizations.

1) Loop Unrolling for Parameter Reuse: In the negacyclic vector-matrix polynomial multiplication (`negacyclic_vector_matrix_polymul`) routine [2], the system independently discovered that unrolling the primary `for` loop by a factor of 8 yielded significant latency reductions. In the baseline trace, each block execution took 12.65 μs. By unrolling the loop, the system enabled the reuse of common parameters across multiple kernels. Because the target parameter size (3.51 MB) fit within the available on-chip memory, this transformation reduced the latency of the first 8 blocks to 1.61 μs. While this represents a standard

engineering optimization, it highlights the system's ability to automatically identify and exploit under-optimized baselines.

2) Fine-Grained Memory and Compute Scheduling: The system optimized data access patterns by adjusting the tiling and scheduling parameters of the kernel. Specifically, it modified the block size parameter from `block_b = 2` to `block_b = 1`. This modification allowed the XLA compiler to schedule more fine-grained data accesses to the accelerator's memory hierarchy. This optimization reduced the overall bootstrapping latency from 7.89 ms to 6.29 ms.

3) Elimination of Redundant Type Casts: The most significant single optimization involved the removal of an explicit bitcast operation. The baseline implementation explicitly cast a right-hand side (`rhs`) operand to 8-bit integers (`i8`). AlphaEvolve identified that the dynamic range of the input data (values from 0 to 255) was already sufficiently captured by the existing `bf16` precision. By removing the redundant and expensive bitcast operator, the system reduced the execution time from 6.29 ms to 3.35 ms. This optimization requires a deep understanding of both the mathematical dynamics of the FHE scheme and the specific hardware execution costs of bitcast operations, making it difficult for human engineers to consistently identify.

V. AlphaEvolve for Optimizing CKKS on TPU

A. Setup

1) Reference code: We use the hand-optimized `CKKS-Rot` and `CKKS-Mult` kernels from CROSS [19], the library offering SotA throughput, as the initial programs and apply AlphaEvolve to search for faster implementations. We expose the implementations of Rescale, TensorMultiply, KeySwitch, ApproxModDown, and AutoMorphism from both kernels to AlphaEvolve.

2) Prompt Setup: We use the system prompt in §A together with a prompt sampled uniformly from the following pool. These prompts are designed to steer AlphaEvolve towards implementation-level improvements, especially scheduling and layout choices that increase VReg utilization in TPU, rather than toward new algorithms discovery.

3) Correctness, and Performance Tests: Functional correctness is enforced using a hidden test set with fixed ciphertext inputs and expected ciphertext outputs. Performance is measured from XProf traces, and we return the negative kernel latency as the optimization score so that lower latency corresponds to a higher score.

For each iteration, AlphaEvolve receives a prompt sampled uniformly from the four prompt templates described in §A. We run the evolution process with 10 controllers and 10 evaluators per controller, for a total of 100 TPUv5e chips.

B. Results

• **`CKKS-Rot`**: On TPUv5e, AlphaEvolve reduces the latency of `CKKS-Rot` from 4874,µs to 3754,µs, yielding a 1.31× speedup. This performance gain is primarily achieved by partitioning a tensor into two smaller sub-tensors. This strategic split triggers specialized XLA optimizations that significantly improve Vector Register (VReg) utilization during the AutoMorphism stage. Consequently, AutoMorphism's contribution to the total end-to-end latency drops from 12% to just 4%. These results demonstrate AlphaEvolve's ability to expose and exploit non-intuitive, compiler-specific performance "sweet spots" that are exceedingly difficult for human engineers to discover manually.

• **`CKKS-Mult`**: On TPUv5e, AlphaEvolve reduces `CKKS-Mult` latency from 6340 µs to 6106 µs, corresponding to a 1.03× speedup. Such performance improvement comes from the scheduling optimization to reuse common parameters. The modest improvement indicates that `CKKS-Mult` is less constrained by underlying compiler inefficiencies.

Fig. 7: `CKKS-Rot` trace comparison before and after evolution (top: 4874 µs; bottom: optimized trace, 3754 µs).

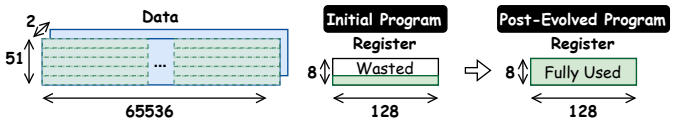

Fig. 8: Breakdown of performance improvements and remaining bottlenecks.

C. Analysis of AlphaEvolve improvements

These results show that AlphaEvolve can improve CKKS kernel performance on TPU by discovering implementation changes that increase effective VReg utilization and reduce inefficiencies in the execution schedule. As shown in Fig. 8, the remaining bottleneck is dominated by unavoidable reshape and reorganization overhead from the current programming APIs, which limits further gains even after compute-side improvements.

VI. Comparison to Related Work

To our knowledge this is the first application of AI technologies to improve cryptographic performance. The closest work for improving kernel generation is done by autocomp [11] and EvoX [14] where agentic infrastructure has been used to improve machine learning kernels.

VII. Conclusion

This paper presents the first application of agentic AI to optimize FHE primitives on TPUs. We detail the design of using AlphaEvolve, a closed-loop system that combines LLMs with real-world hardware execution feedback and rigorous correctness checks. We demonstrate the framework's ability to autonomously discover performance improvements on Google's mainstream ML accelerators.

Our results demonstrate that AI-driven search uncovers optimizations frequently missed by domain experts. Although our initial results focus on TPUs, the framework is highly extensible. This methodology can potentially optimize FHE for any hardware accelerator, or be integrated with simulators to guide specialized hardware-software co-design.

A. Experience and Lessons Learned using AlphaEvolve

Some of our experiences and learnings in adapting AlphaEvolve for cryptographic optimization are described below:

1) Granularity of Optimization: Targeting isolated operations, such as polynomial multiplication (`polymul` [2]), failed to produce end-to-end speedups. We found that the agent requires visibility into broader operational scopes to make a meaningful impact on overall latency.

2) Co-evolution and System-Level Scoring: Building on the need for broader visibility, shifting to the co-evolution of the vector-matrix product and scoring it against the complete bootstrapping execution successfully reduced latency from 10 ms to 5.6 ms.

3) Guided Domain Constraints: While evolving the `blind-rotate` operation [2], the system discovered that RLWE decomposition allowed input tensors to safely use `int8` rather than `int32`, and that unrolling the loop by a factor of eight yielded significant benefits. This underscores the importance of domain experts selectively granting the AI permission to modify high-impact code blocks.

4) Comprehensive Correctness Tests: Initial attempts to evolve the AND-gate failed to cover all values of a 3-input lookup table (`lut3` [2]). We had to update the evaluator to exhaustively verify all lookup table values, demonstrating that strict, comprehensive validation is essential to prevent the model from reward hacking.

5) Security Checks: Because performance cannot come at the expense of cryptographic integrity, we introduced a standardized 128-bit security parameter check using a lattice estimator when co-optimizing scheme parameters.

6) Feedback and Prompts: To effectively steer the evolutionary search, we integrated hardware profiling tools (`xprof`) directly into the feedback loop and provided hardware-specific optimization guidelines as automated compilation prompts.

B. Future Work

Our initial results lay the groundwork for the next generation of automated cryptographic hardware-software co-design and opens several avenues for future research. To ensure security guarantees are maintained during hardware optimization, future iterations will integrate parameter selection with noise tracking. Furthermore, the bottleneck of manual code reviews can be reduced by adding functional equivalence checkers directly into the validation pipeline. Finally, a fundamental open question remains in the design of reward functions to safely allow AI agents to invent "novel" cryptographic algorithms and protocols while preserving security.

ACKNOWLEDGMENTS

The authors express their gratitude to Andrew Ferraiuolo, Ben Kreuter, and Cindee Madison for their valuable feedback during the initial phases of this work. We thank Po-Sen Huang, Ngân (NV) Vũ, and the AlphaEvolve team for their assistance in establishing the experimental framework. We thank Bryant Gipson, Amanda Walker, Moti Yung, and Elie Burzstein for their leadership and guidance throughout this project.

We also acknowledge the use of large language models (LLMs) in the preparation of this manuscript. Specifically, the models were used for proofreading, summarizing text, and generating LaTeX outlines. The authors have reviewed and verified all generated content and take full responsibility for the final manuscript.

REFERENCES

[1] M. R. Albrecht, R. Player, and S. Scott, "On the concrete hardness of learning with errors," Cryptology ePrint Archive, Paper 2015/046, 2015. [Online]. Available: https://eprint.iacr.org/2015/046

[2] A. Ali, E. Astor, B. Gipson, S. Gorantala, M. Guevara, J. Kun, W. Lam, R. Misoczki, R. Springer, J. Takeshita, J. Tong, C. Tew, and C. Yun, "Jaxite: A fully homomorphic encryption backend targeting TPUs and GPUs, written in JAX," https://github.com/google/jaxite, 2024.

[3] A. Ali, J. Choi, B. Gipson, S. Gorantala, J. Kun, W. Legiest, L. Lim, A. Viand, M. Z. Demissie, and H. Zheng, "Heir: A universal compiler for homomorphic encryption," 2025. [Online]. Available: https://arxiv.org/abs/2508.11095

[4] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018. [Online]. Available: http://github.com/jax-ml/jax

[5] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Advances in Cryptology – ASIACRYPT 2017*, ser. Lecture Notes in Computer Science, vol. 10624. Springer, 2017, pp. 409–437.

[6] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "TFHE: Fast fully homomorphic encryption over the torus," *Journal of Cryptology*, vol. 33, no. 1, pp. 34–91, Jan 2020.

[7] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing*, ser. STOC '09. New York, NY, USA: Association for Computing Machinery, 2009, p. 169–178. [Online]. Available: https://doi.org/10.1145/1536414.1536440

[8] S. Gorantala, R. Springer, and B. Gipson, "Unlocking the potential of fully homomorphic encryption," *Communications of the ACM*, vol. 66, no. 6, pp. 68–75, jun 2023. [Online]. Available: https://doi.org/10.1145/3582500

[9] S. Gorantala, R. Springer, S. Purser-Haskell, W. Lam, R. J. Wilson, A. Ali, E. P. Astor, I. Zukerman, S. Ruth, C. Dibak, P. Schoppmann, S. Kulankhina, A. Forget, D. Marn, C. Tew, R. Misoczki, B. Guillen, X. Ye, D. Kraft, D. Desfontaines, A. Krishnamurthy, M. Guevara, I. M. Perera, Y. Sushko, and B. Gipson, "A general purpose transpiler for fully homomorphic encryption," *CoRR*, vol. abs/2106.07893, 2021. [Online]. Available: https://arxiv.org/abs/2106.07893

[10] X. He, "Accelerated linear algebra compiler for computationally efficient numerical models: Success and potential area of improvement," *PLOS ONE*, vol. 18, no. 2, p. e0282265, 2023.

[11] C. Hong, S. Bhatia, A. Cheung, and Y. S. Shao, "Autocomp: A powerful and portable code optimizer for tensor accelerators," 2025. [Online]. Available: https://arxiv.org/abs/2505.18574

[12] JAX Developers, "Pallas quickstart — JAX documentation," https://docs.jax.dev/en/latest/pallas/quickstart.html, 2024.

[13] N. P. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles, C. Young, X. Zhou, Z. Zhou, and D. Patterson, "Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings," 2023. [Online]. Available: https://arxiv.org/abs/2304.01433

[14] S. Liu, S. Agarwal, M. Maheswaran, M. Cemri, Z. Li, Q. Mang, A. Naren, E. Boneh, A. Cheng, M. Z. Pan, A. Du, K. Keutzer, A. Cheung, A. G. Dimakis, K. Sen, M. Zaharia, and I. Stoica, "Evox: Meta-evolution for automated discovery," 2026. [Online]. Available: https://arxiv.org/abs/2602.23413
[15] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. Ruiz, A. Mehrabian *et al.*, "Alphaevolve: A coding agent for scientific and algorithmic discovery," *arXiv preprint arXiv:2506.13131*, 2025.
[16] OpenXLA Project, "Profiling JAX computations with XProf," https://openxla.org/xprof/jax_profiling, 2024.
[17] OpenXLA Project, "OpenXLA: Accelerated linear algebra," https://github.com/openxla/xla, 2026.
[18] J. Tong, J. Dang, S. Langowski, T. Huang, A. Ali, J. Kun, S. Devadas, and T. Krishna, "Morph: Enabling ai asics for zero knowledge proof," in *Proceedings of the 63nd Annual ACM/IEEE Design Automation Conference*, ser. DAC '26, 2026.
[19] J. Tong, T. Huang, J. Dang, L. de Castro, A. Itagi, A. Golder, A. Ali, J. Jiang, J. Kun, Arvind, G. E. Suh, and T. Krishna, "Leveraging asic ai chips for homomorphic encryption," ser. HPCA'26. Australia: 2026 IEEE International Symposium on High Performance Computer Architecture (HPCA), 2026.

APPENDIX

A. LLM Instruction-Prompt Pool

1) **Prompt 1:** Suggest a new way to minimize data reorganization in the program.
2) **Prompt 2:** Suggest a new way to keep the logical shapes of JAX arrays invariant throughout execution.
3) **Prompt 3:** Suggest a new way to hide off-chip memory latency behind computation.
4) **Prompt 4:** Suggest a new optimization idea based on your expert knowledge of TPU performance tuning.

B. Role and Hardware Architecture Prompts

Role: You are an expert hardware-aware compiler optimizer specializing in TPU architectures.
Objective: Optimize the input computational graph for maximum performance on the TPU v5e architecture. You must make strategic decisions regarding operation mapping (Vector vs. Matrix units), memory hierarchy management, and parallelism.
Target Hardware Specifications: Use the following parameters:

- **Compute Capabilities (Peak):**
  - Clock Frequency: 1.50 GHz
  - Matrix Units (MXU): 197 TFLOPS (`bf16`/`fp8`), 394 TOPS (`int8`), 788 TOPS (`int4`).
  - Vector Units (VPU): `vmatmul` (1.54 TB/s), `vmatpush` (6.16 TB/s).
- **Memory Constraints (CRITICAL):**
  - `CMEM`: None (0 bytes). Do not tile data into CMEM.
  - `Vmem` (Vector Memory): 128 MB. Read: 18.5 TB/s, Write: 6.16 TB/s.
  - `HBM` (High Bandwidth Memory): ∼17.2 GB at 820 GB/s.
- **Interconnect (ICI):** 2D Torus, 4 links/chip, 45 GB/s per link.

Optimization Strategies Required:
- **Compute Mapping:** Map matrix multiplications to the Systolic Array (MXU). Map element-wise, reduction, and non-matmul ops to the Vector Unit (VPU).
- **Memory Tiling & Fusion:** All functional tiling must target the 128 MB Vmem. Perform loop fusion to avoid spilling back to HBM.
- **Precision Tuning:** Evaluate suitability for `int8` quantization for 2× throughput.
- **Pipeline Parallelism:** Overlap HBM-to-Vmem DMA transfers with VPU/MXU execution.

Output Requirements: Generate optimized code/IR and summarize: theoretical peak utilization, memory bandwidth utilization, and a bottleneck analysis.